

A Linear Reduction Model for Parallel *Prolog* System

IEEE 4TH INTERNATIONAL CONFERENCE ON CLOUD
COMPUTING AND BIG DATA ANALYTICS

BY FELIPE CARVALHO, GUSTAVO DIAS, THIAGO GOMES





- INFORMAÇÕES GERAIS.....03
- CONTEXTUALIZAÇÃO.....04
- INTRODUÇÃO.....05
- PROPOSTA GERAL.....06
- PROSTA DO ARTIGO.....07
- MECANISMO DE RACIOCÍNIO PARALELO.....10
- ESTRATÉGIAS DE BUSCA.....11
- OBJETIVO E BENEFÍCIOS DO PIE.....12
- CONCLUSÃO.....13
- DESTAQUE.....14
- REFERÊNCIAS BIBLIOGRAFICAS.....18

Informações Gerais “

TÍTULO: A Linear Reduction Model for Parallel Prolog System

CONFERÊNCIA: IEEE 4th International Conference on Cloud Computing and Big Data Analytics.

AUTOR(ES): Wei Zhang, Shenggui Hong.

DOI: 978-1-7281-1410-1

ANO: 2019

PUBLICADOR: IEEE

LOCAL: Shenyang, China

Contextualização

CONTEXTUALIZAÇÃO:

- O artigo enquadra o problema no campo da Inteligência Artificial Distribuída (DAI);
- Prolog é uma famosa linguagem de programação lógica, e ele pode expressar naturalmente o pensamento e o raciocínio humano, ele tem sido usado em aplicação de inteligência artificial;
- Prolog elimina as descrições procedimentais da programação, ele oferece a possibilidade de um grande número de execuções concorrentes.

PRINCIPAIS PROBLEMAS:

1. A incerteza do Prolog gera muitos retrocessos;
2. O processo de execução do Prolog precisa implementar a correspondência de padrões em dados de grande escala .

Introdução

ÁRVORE OR:

É completa, mas não explora paralelismo AND (ou seja, os sub objetivos que poderiam ser resolvidos em paralelo acabam sendo processados sequencialmente);

ÁRVORE OR/AND:

Permite o paralelismo AND e OR, mas exige operações de interseção sobre os retornos de variáveis comuns entre subproblemas. Essas operações (retornar para interseção, abandonar soluções parciais quando a interseção é vazia, e libertar/redefinir bindings de variáveis) são complexas e caras.

- (1) ? - Fallible(x),Greek(x);
- (2) Fallible:- Human(x);
- (3) Human(Turing) :- ;
- (4) Human(Socrates) :-;
- (5) Greek(Socrates) :- ;

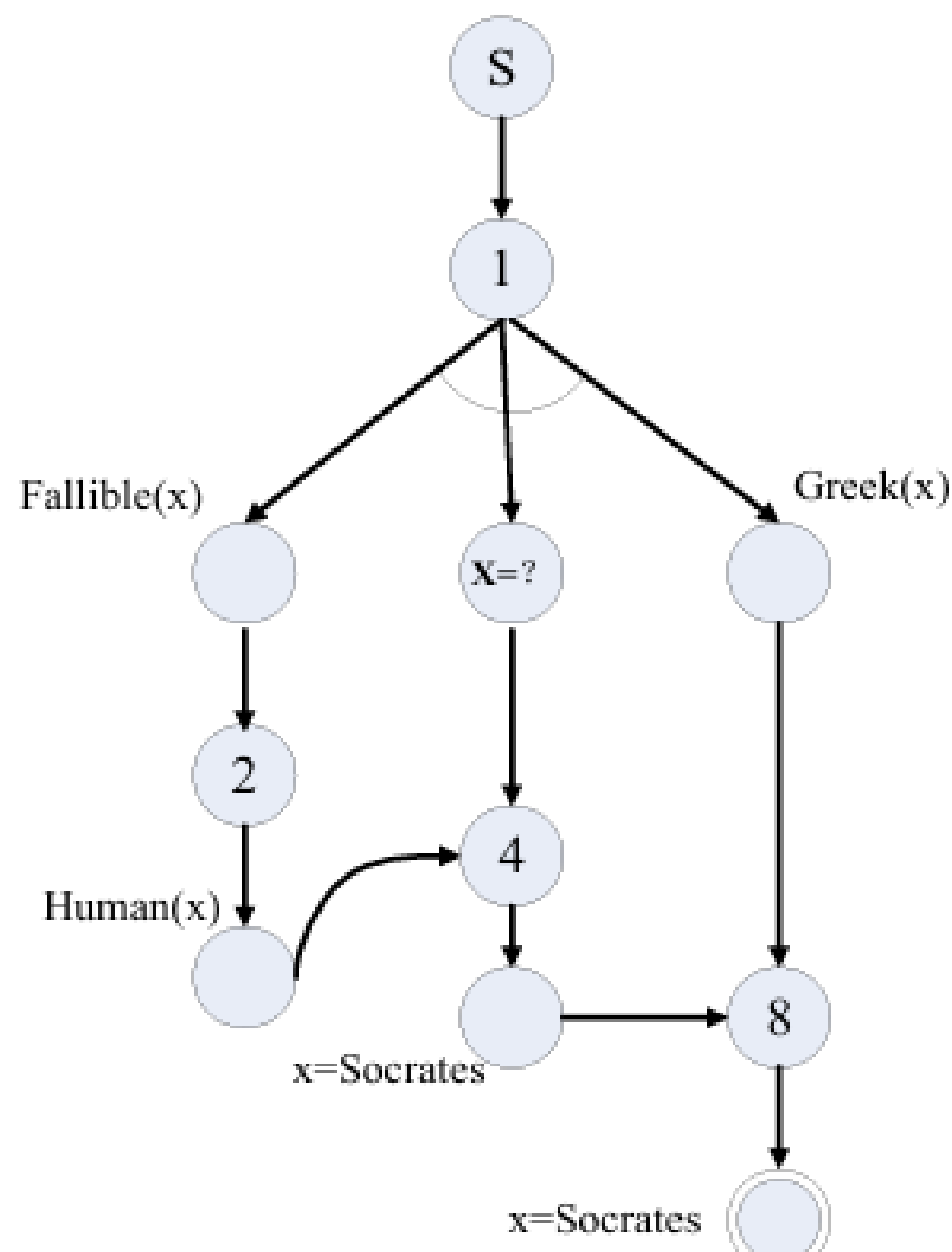
Proposta

REQUISITOS DE UM BOM MODELO DE DERIVAÇÃO:

- Confiabilidade e completude;
- Explorar plenamente OR e AND paralelismo;
- Boas complexidades de tempo e espaço para o algoritmo de busca.

PROPOSTA CONCEITUAL:

- Como isso, os autores propõem usar formalmente o Generalized AND/OR Graphs (GAG), como modelo de raciocínio;
- É um grafo dirigido, ou seja, subproblemas podem se conectar e pode expressar vínculos e restrições entre variáveis diretamente nos nós e arestas do grafo.



Proposta DO ARTIGO

Substituir o backtracking por um modelo de busca em grafos podendo ser executado em paralelo

Grafos Generalizados AND/OR (GAGs)

Paralelismo AND & OR

Algoritmos de Busca no GAG:

Algoritmo Ao:

- Busca heurística básica, admissível, mas consome muita memória.

Algoritmo AO.A*:

- Adaptação do A* para GAGs.
- Usa função heurística para priorizar caminhos promissores.
- Mais eficiente em tempo e memória.

Proposta DO ARTIGO

Grafos Generalizados AND/OR (GAGs) + Paralelismo

O GAG É UM GRAFO QUE REPRESENTA TODAS AS POSSÍVEIS DERIVAÇÕES DO PROGRAMA PROLOG, INCLUINDO AS RESTRIÇÕES ENTRE VARIÁVEIS. CADA NÓ NO GAG PODE SER:

- **NÓ AND:** REPRESENTA UM CONJUNTO DE SUB-OBJETIVOS QUE DEVEM SER TODOS SATISFEITOS
- **NÓ OR:** REPRESENTA ALTERNATIVAS DIFERENTES PARA SATISFAZER UM OBJETIVO
- **NÓ DE RESTRIÇÃO:** REPRESENTA EXPLICITAMENTE AS LIGAÇÕES DE VARIÁVEIS

MUDANÇA SINTÁTICA:

ANTES: IMPLÍCITO
HUMAN(X) :- FALLIBLE(X).

DEPOIS: EXPLÍCITO
HUMAN(X), X=? :- FALLIBLE(X), X=?.

Proposta DO ARTIGO

Algoritmos de Busca no GAG

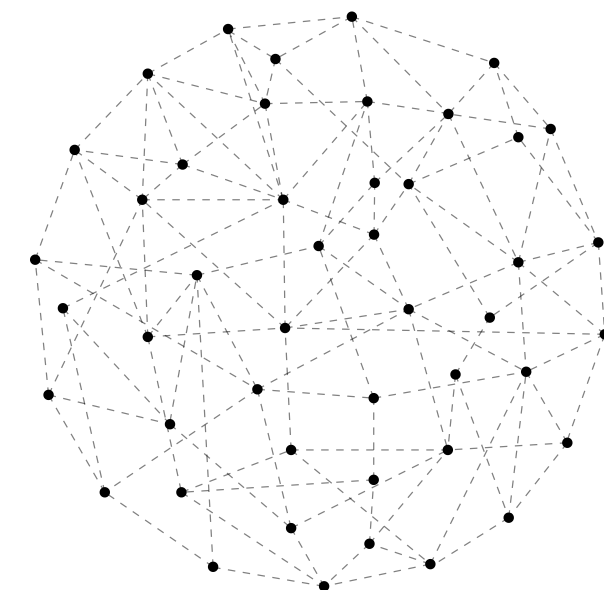
ALGORITMOS QUE DECIDEM
O MELHOR CAMINHO PARA
UMA BUSCA

ALGORITMO AO:

1. Começa com o nó inicial no "open list"
2. Enquanto houver nós para expandir:
 - Pega o nó com menor valor heurístico (h-value)
 - Se for solução → termina com sucesso
 - Expande gerando todos os descendentes
 - Atualiza listas open/closed
3. Se open list ficar vazia → falha

ALGORITMO AO.A*:

1. Usa função de avaliação $f(n) = g(n) + h(n)$
 - $g(n)$: custo acumulado até n
 - $h(n)$: estimativa heurística do custo até a solução
2. Mantém apenas o MELHOR caminho para cada conjunto terminal (tns)
3. Se encontra um caminho melhor para um nó já visitado, atualiza



Mecanismo de Raciocínio Paralelo

O QUE É O PIE?

- Baseado no modelo GAG;
- Fornece o usuário uma única imagem;
- Composto por n motores paralelos;

ESTRUTURA DO PIE

- Matcher (Correspondente):
Determina quais cláusulas podem ser executadas;
- Arbiter (Árbitro):
Decide qual pie executará qual função;
- Action (Ação):
Responsável por expandir a árvore e localizar os próximos sub alvos
- Common Data Área:
Armazena a rede de padrões das regras;

Estratégias de Busca

PARALELISMO OR → Execução simultânea de alternativas de regras.

PARALELISMO AND → Execução simultânea de subproblemas independentes dentro de uma mesma regra.

É POSSIVEL ALTERAR O SISTEMA DE BUSCA?

Objetivo e Benefícios do PIE

- Eliminar o backtracking tradicional do Prolog, substituindo-o por busca linear em grafos GAG;
- Permitir execução paralela real, tanto entre alternativas (OR) quanto entre subproblemas (AND);
- Melhorar drasticamente a eficiência da dedução Prolog, reduzindo redundâncias e custos de interseção de variáveis;
- Aproveitar técnicas maduras de busca heurística (como A^* e $AO.A^*$) diretamente sobre o grafo GAG;
- Transforma o raciocínio lógico de Prolog em um processo de busca sem retrocesso, totalmente paralelizável e com grande ganho de desempenho;

Conclusão

RESULTADOS E CONCLUSÃO DO ARTIGO:

A conclusão mostra que, apesar de várias tentativas (árvores OR, AND/OR e híbridos), o processo de inferência em Prolog continua ineficiente na prática por causa do volume de backtracking exigido.

CONCLUSÃO DO GRUPO:

Com isso, chegamos a conclusão que eles foram muito ambiciosos em tentar resolver um dos principais problemas do Prolog, a ineficiência causada pelo backtracking. Eles tiveram ideias muito boas, um embasamento teórico excelente, mesmo não conseguindo atingir o objetivo, ajudaram muito na pesquisa na área de linguagens lógicas e de Prolog.

Destaque

FELIPE

Abordagem do artigo é interessante, com uma boa explicação da proposta assim como os exemplos. Como primeiro contato foi difícil de entender porque se trata de uma linguagem não habitual para mim e além do artigo abordar questões de grafos que ainda não tenho certo conhecimento. Mas achei muito contribuidor ele ter desenvolvido algoritmos de fato para a solução e não ficou só na pesquisa teórica.

Destaque

GUSTAVO

A parte mais interessante do artigo é a grande quantidade de benefícios que são trazidos para linguagem com a implementação do PIE e como o artigo é bem fundamentado na parte teórica. No primeiro contato é difícil de entender para quem não é experiente com Prolog, se tornando um artigo totalmente nichado para quem programa na linguagem, porém, isso é exatamente o que dá margem para um referencial teórico mais profundo e sólido.

Destaque

THIAGO

A parte que considero mais interessante no artigo é a utilização dos grafos generalizados (GAG) como solução para as limitações dos modelos tradicionais baseados em árvores. Essa proposta representa um avanço conceitual importante, pois substitui a estrutura hierárquica das árvores de derivação AND/OR por uma representação gráfica mais flexível e expressiva, que consegue mostrar explicitamente as relações e restrições entre as variáveis. As árvores de derivação obrigam o sistema Prolog a percorrer caminhos sequenciais e a recorrer constantemente ao retrocesso (backtracking) para corrigir escolhas inválidas, o uso de grafos permite transformar o raciocínio em um processo de busca sem retrocesso. Essa mudança elimina a necessidade de explorar as subárvores várias vezes, tornando o processo de dedução mais direto, rápido, eficiente e naturalmente paralelizável.

Sem mais **PROLOnGações**....

Muito Obrigado!



Referências Bibliográficas

- ZHANG, Wei; HONG, Shenggui. A Linear Reduction Model for Parallel Prolog System. In: IEEE 4th International Conference on Cloud Computing and Big Data Analytics, 2019, Shenyang, China. [S. l.: s. n.], 2019.

